

TRABAJO PRÁCTICO N.º 1

Perceptrón simple

Materia: Redes neuronales

Alumnos: Thiago García

Docente: Emiliano

Fecha de entrega: 28 de abril de 2026

1. Implemente el algoritmo de perceptron simple con función de activación escalón y utilícelo para aprender los siguientes problemas:

- Función lógica 'Y' con entradas $x = \{-1, 1\}, \{1, -1\}, \{-1, -1\}, \{1, 1\}$, y salida esperada $y = \{-1, -1, -1, 1\}$.
- Función lógica 'O exclusivo' con entradas $x = \{-1, 1\}, \{1, -1\}, \{-1, -1\}, \{1, 1\}$, y salida esperada $y = \{1, 1, -1, -1\}$.

¿Qué puede decir acerca de los problemas que puede resolver el perceptron simple escalón en relación a la resolución de los problemas que se le pidió que haga que el perceptron aprenda?

2. Implemente el algoritmo del perceptron simple lineal y perceptron simple no lineal y utilícelos para aprender el problema especificado en los archivos TP1-ej2-Conjuntoentrenamiento.txt y TP1-ej2-Salida-deseada.txt. • Evalúe la capacidad del perceptron simple lineal y perceptron simple no lineal para aprender la función cuyas muestras están presentes en los archivos indicados.

- Evalúe la capacidad de generalización del perceptron simple no lineal utilizando, de los datos provistos, un subconjunto de ellos para entrenar y otro subconjunto para testear.
- ¿Cómo podría escoger el mejor conjunto de entrenamiento?
- ¿Cómo podría evaluar la máxima capacidad de generalización del perceptron para este conjunto de datos?

Presentacion: El trabajo deberá ser presentado por cada grupo el día 20 de abril. Para su presentación usar powerpoint o cualquier programa similar. En la presentación deberá figurar el título del trabajo, el nombre de la materia, el nombre de los integrantes del grupo y la fecha. La presentación podría estar dividida de acuerdo a los items solicitados indicando en cada uno que es lo que se le pide. Para cada item, primero comentar lo que se hizo y las decisiones tomadas para llevarlo a cabo (si correspondiera). Luego exponer las dificultades que se presentaron (si correspondiera) y finalmente exponer los resultados. Al finalizar la presentación deberán exponerse las conclusiones del trabajo.

Respuestas

1.

Implementacion:

Función lógica 'Y' : Implementamos el algoritmo perceptrón simple con activación escalón. Con 4 muestras y salidas -1/1, el perceptrón encontró pesos que clasifican perfectamente en solo 9 iteraciones. Error final = 0.

Función lógica 'O exclusivo' : Mismo código, solo cambia y. El error nunca llega a 0 , oscila indefinidamente. El perceptrón no puede resolverlo porque el XOR no es linealmente separable (no existe una recta que separe las clases en 2D).

Pregunta 1:

Función Y: el error llegó a 0 en 9 iteraciones → el perceptrón encontró un hiperplano que separa perfectamente las 4 clases. Es linealmente separable.

XOR: el error nunca baja de 1, oscila entre 1-3 durante 1000 iteraciones sin estabilizarse → el perceptrón no puede encontrar ese hiperplano porque no existe. En 2D, no hay una sola recta que separe $\{(-1,1),(1,-1)\}$ de $\{(-1,-1),(1,1)\}$.

"El perceptrón escalón busca un hiperplano separador. En la Y lo encontró (error=0). En el XOR el gráfico muestra que el error nunca converge, eso no es un problema del algoritmo ni de los parámetros, sino que geoméricamente es imposible separar esos puntos con una sola recta."

2.

Implementacion:

Lineal: Mismo algoritmo pero con activación $O = h$ y error MSE. Con 200 muestras de 3 dimensiones y salidas reales (0-99), converge a $MSE \approx 9257$ y no baja más, indica que la relación en los datos no es puramente lineal.

"Gráfico: salida esperada (azul) vs predicción del perceptrón (naranja punteado), 200 muestras en eje X, valor real en eje Y (escala 0-99). El modelo sigue la tendencia general pero con errores notables: baja en los picos altos y predice valores negativos donde no debería. Refleja la limitación del modelo lineal."

No lineal: Mismo algoritmo pero con activación $O = \tanh(h)$ y Δw incluye la derivada de $\tanh$. Las salidas se normalizan a $(-1,1)$ antes de entrenar. Con 200 muestras de 3 dimensiones, converge a $MSE \approx 4.6$, mucho menor que el lineal, lo que confirma que la función tiene una relación no lineal que este modelo sí puede capturar.

"Gráfico: salida esperada (azul) vs predicción del perceptrón (naranja punteado), 200 muestras en eje X, valor real en eje Y (escala 0-99, desnormalizado desde tanh). El modelo sigue de cerca la curva real en casi todas las muestras. La superposición de ambas líneas indica un buen aprendizaje de la función."

Evaluacion:

Se dividieron los 200 datos en 80% entrenamiento (160 muestras) y 20% test (40 muestras). El modelo fue entrenado únicamente con las 160 muestras y luego evaluado con las 40 que nunca vio. El error de entrenamiento ($MSE \approx 3.78$) y el error de test ($MSE \approx 1.03$) son similares y bajos, lo que indica que el modelo generaliza bien, no memorizó los datos sino que aprendió la función subyacente.

"La línea roja separa la zona de entrenamiento (izquierda) de la zona de test (derecha). Se observa que la predicción (naranja) sigue de cerca la salida esperada (azul) en ambas zonas, confirmando la capacidad de generalización del perceptrón no lineal."

Pregunta 1:

El conjunto de entrenamiento ideal es aquel que representa de forma equilibrada toda la distribución de los datos. En nuestra implementación, la división 80/20 se realizó tomando los primeros 160 datos en orden secuencial, lo que puede introducir un sesgo si los datos tienen algún patrón de ordenamiento.

Para obtener el mejor conjunto de entrenamiento se podría aplicar mezcla aleatoria previa al corte (`np.random.permutation`), de modo que train y test sean muestras representativas del dataset completo y no

dependan del orden original. Una estrategia más robusta consiste en repetir la división con distintas semillas aleatorias y seleccionar el corte cuyo error de test sea mínimo , garantizando que el conjunto de entrenamiento cubra la mayor variedad posible de la función subyacente.

Pregunta 2:

La forma estándar de evaluar la capacidad de generalización de manera más confiable es la validación cruzada de K iteraciones (K-Fold Cross-Validation). El procedimiento consiste en dividir los 200 datos en K subconjuntos iguales. En cada iteración, K-1 subconjuntos se usan para entrenar y el restante para testear; el resultado final es el promedio de los K errores de test obtenidos.

Con K=5 se obtienen exactamente 5 cortes de 160/40, similares al que realizamos, pero evaluados sobre distintas particiones del dataset. Si el MSE promedio resultante es consistentemente bajo y similar al error de entrenamiento, se puede concluir que el modelo alcanzó su máxima capacidad de generalización para estos datos , es decir, aprendió la función subyacente y no simplemente memorizó las muestras de entrenamiento.